Speaker Script

Suleyman's Sections

CAS 760 Development Project 2

A Simple File System in Alonzo

Target timing: ∼7 minutes total, ∼60 seconds per slide.

My slides, in order (deck slides 8–14):

1. Theory for Files — FILE
2. Theory Extension for Secured Files — SECURED-FILE
3. Theory Extension for Directories — DIRECTORY
4. Directories as an Inductive Type
5. Directory Development — DIR-DEV (both `contains` and `size` defined here)
6. Explanation for Directory Development
7. Example of Directory Evaluation (handoff to Hamza)

How to read this:

- The *italic indented text* is what I say **verbatim**.
- → **Red arrows** are stage directions — where to point, gesture, or pause.
- *(pause)* means take a breath, about one second.
- ⇒ **Green TRANSITION** means advance to the next slide.
- Each slide starts on a fresh page — flip as you go.

Read this aloud at least twice before the talk. Time yourself.

SLIDE 1 · Files — Theory FILE

timing: ∼70 sec

ON SCREEN: *Base types* N, C, F. *Constants* mkfile, fname, fcontent *with types. Seven axioms (1–7) listed.*

"Thanks Hamza. Now I'll walk through the Files theory."

→ **Brief nod toward Hamza. Face the audience.**

"The idea here is straightforward: a file is a named container for content. Conceptually, a file is just a pair — a name and some content — but we want to formalize that abstractly, without committing to what names or contents actually are."

"So we introduce three base types —"

→ **Point to the line "N, C, F" on the slide.**

"— N for names, C for content, and F for the file type itself. We keep them opaque on purpose, so that this theory captures the ***structure*** *of files rather than any specific representation."*

(pause)

"We have one constructor, mkfile *—"*

→ **Point to the** mkfile **line.**

"— of type $N \to C \to F$, which builds a file from a name and content. And we have two accessors, fname *and* fcontent *—"*

→ **Point to the** fname **and** fcontent **lines in turn.**

"— that pull the name and content back out."

"The axioms say what you'd expect. Axioms 1 and 2: mkfile *is total and injective — every name-content pair gives a valid file, and distinct pairs give distinct files. Axioms 3 and 4 are the two accessor axioms:*

if you build a file with mkfile *n c, extracting the name gives you n back, and the content gives you c. Axioms 5 and 6 say the accessors are themselves total. And Axiom 7 — surjectivity — says every file f is equal to* mkfile *applied to its own name and content. That closes the loop: F is isomorphic to $N \times C$, but presented abstractly as a theory."*

⇒ **TRANSITION:** advance to Secured Files slide.

SLIDE 2 · Secured Files — Theory Extension SECURED-FILE

timing: ∼75 sec

ON SCREEN: *"Extends: FILE and PERM". "New base types: none (P inherited from PERM)". New constants* perm, protect. *Axioms 8–12.*

> *"SECURED-FILE combines files and permissions. This is one of the more interesting design points in the paper."*

> *"The intuition is: FILE plus PERM equals SECURED-FILE. Each file gets a permission level attached to it. Formally, we present SECURED-FILE as a* ***joint extension*** *of both FILE and PERM —"*

→ **Point to the "Extends: FILE and PERM" line on the slide.**

> *"— this is a pushout-style construction in the little theories method."*

> *"What that means concretely: SECURED-FILE has* ***no new base types*** *of its own."*

→ **Point to "New base types: none" line.**

> *"It inherits P from PERM and inherits N, C, F from FILE. The two extension arrows in the theory graph — from PERM to SECURED-FILE, and from FILE to SECURED-FILE — encode that inheritance formally."*

(pause)

> *"What SECURED-FILE* ***does*** *add is two new constants."*

→ **Point to** perm **line.**

> *"First,* perm*, of type $F \to P$, which tells you the permission of a file."*

→ **Point to** protect **line.**

"And second, protect*, of type $F \to P \to F$, which takes a file and a permission and gives you back the same file with that permission assigned."*

"Axioms 8 through 12 govern these. Eight and nine are totality. Ten is the core semantics: applying protect *with permission p gives a file whose permission is p. And eleven and twelve say that* protect *only changes the permission — it preserves the name and the content. So* protect *is a pure permission-update operation."*

⇒ **TRANSITION:** advance to Directories slide.

SLIDE 3 · Directories — Theory Extension DIRECTORY

timing: ∼70 sec

ON SCREEN: *"Design choice: flat structure." Base type D. Constructors* emptydir, addfile. *Axioms 13–16.*

> *"DIRECTORY is a theory extension over SECURED-FILE. This is where the design choice I want to highlight comes in: we model directories as a* ***flat*** *structure, not as a hierarchical tree."*

> *"The reason is that this keeps the inductive type clean."*

→ **Point to the** addfile **type signature on the slide.**

> *"The constructor* addfile *has type $F \to D \to D$ — it takes a* ***file****, not a directory, as its first argument. That gives us a direct analogy with the list type from the textbook:* emptydir *plays the role of* nil*, and* addfile *plays the role of* cons*."*

> *"If we wanted a hierarchical model, we'd need a second constructor,* adddir *of type $D \to D \to D$ —"*

→ **If whiteboard available: quickly write "`adddir : D → D → D`" in the corner. Otherwise just emphasize with hand gesture.**

> *"— which would make this a tree-shaped inductive type. That's a natural extension, and we mention it as future work in the paper, but for this project the flat model is the right scope."*

(pause)

> *"The axioms are standard. Axiom 13 says* addfile *is total. Axiom 14 says it's* ***injective*** *— and we'll see later that this injectivity is what makes the FILE-to-DIR morphism work."*

→ **Slight emphasis on "injective" — this callback matters for Hamza's morphism section.**

"Axiom 15 is disjointness: addfile *f d is never equal to* emptydir*. And Axiom 16 is the structural induction principle, which lets us prove properties over all directories by handling the two constructor cases."*

⇒ **TRANSITION:** advance to Inductive Type verification slide.

SLIDE 4 · Directories as an Inductive Type

timing: ∼70 sec

ON SCREEN: $K_D = \{\text{emptydir}, (\text{addfile}, U_F)\}$. *Interpretation bullets. Conclusion about finite lists of* D_F^M.

> *"To confirm DIRECTORY is a proper recursive inductive type in Alonzo, we follow the procedure from Chapter 10 of Farmer's textbook."*

> *"The constructor set is* K_D *—"*

→ **Point to the $K_D = \{\ldots\}$ expression on the slide.**

> *"— which contains* emptydir *and the pair* $(\text{addfile}, U_F)$. emptydir *is a* ***first-kind*** *constructor — it's nullary. And* $(\text{addfile}, U_F)$ *is a* ***fourth-kind*** *constructor: the* U_F *means the first argument ranges over the entire domain of* F*, and because* D *appears in the domain of* addfile*'s type, it makes* addfile *recursive."*

(pause)

> *"We check the seven obligations from Chapter 10 in the paper. I'll just mention them briefly: the quasitype for the* F*-argument is non-empty because* F *is a base type; totality and injectivity hold by Axioms 13 and 14;* emptydir *is not in the range of* addfile *by disjointness; disjoint-ranges is trivial since there's only one non-nullary constructor; and structural induction is Axiom 16. All seven pass."*

> *"The consequence, by the Inductive Types Theorem, is that in every standard model* D *is isomorphic to the set of* ***finite lists*** *over the interpretation of* F*. So directories really do behave like lists of files — exactly as the constructor signatures suggested."*

⇒ **TRANSITION:** advance to DIR-DEV intro slide.

SLIDE 5 · Directory Development — DIR-DEV

timing: ∼90 sec

ON SCREEN: *Name: DIR-DEV. Bottom theory: DIR. Def 1 (*contains*) with Thms 1–2. Def 2 (*size*) with Thms 3–6. All on one slide.*

> *"DIR-DEV is a* ***development*** *over DIRECTORY — not another theory extension. That matters because* `contains` *and* `size` *are* ***derived*** *operations, definable from* addfile *and* emptydir*. So we* ***prove*** *they exist rather than postulating them as new theory constants. In Project 1 we lost points for making the opposite mistake — this is the direct correction."*

→ **Brief pause on the "lesson learned" beat — earns goodwill.**

> *"The slide gives both definitions and all six theorems. I'll walk through them."*

→ **Point to Def 1 (`contains`) on the slide.**

> *"Def 1:* `contains` *of type* $N \to D \to \mathbb{B}$*. Defined as the unique function* g *satisfying two recursion clauses —* $g\ n$ emptydir *is false, and* $g\ n$ (addfile $f\ d$) *is true if* fname f *equals* n*, or if* $g\ n\ d$ *is already true. So* `contains` *either matches the outermost file's name, or recurses into the tail."*

→ **Move pointer to Thms 1–2.**

> *"Theorem 1 says* `contains` *is total. Theorem 2 says the empty directory contains no name."*

(pause)

→ **Point to Def 2 (`size`) on the slide.**

> *"Def 2:* `size` *of type* $D \to \mathbb{N}$*. Same iota-term pattern —* g emptydir *equals zero, and* g (addfile $f\ d$) *equals* $g\ d$ *plus one."*

→ **Move pointer down the four theorems 3–6.**

"Four theorems: Theorem 3, totality. Theorem 4, size of empty is zero. Theorem 5, the recurrence — `size` (addfile f d) *equals* `size` d *plus one. And Theorem 6, non-negativity, which follows from the recurrence by structural induction. Both definitions are validated in the appendix."*

⇒ **TRANSITION:** advance to the Explanation slide.

SLIDE 6 · Explanation for Directory Development

timing: ∼50 sec

ON SCREEN: *Plain-language prose summary of* `contains` *and* `size`*. Two bullet blocks. Mentions appendix validation.*

> *"This slide restates the same two definitions in plain language for intuition."*

→ **Point to the `Contains` block on the slide.**

> *"*`contains` *n d asks: is there a file named n somewhere in directory d? The empty directory contains nothing; and* addfile *f d contains n exactly when either f's own name is n, or d already contained n."*

→ **Point to the `Size` block.**

> *"*`size` *d counts file entries: zero for* emptydir*, and one more than* `size` *d whenever we prepend a file with* addfile*."*

(pause)

> *"Both definitions are iota-terms — definite descriptions. The appendix discharges existence and uniqueness for each, which is what makes them legal development-level abbreviations."*

⇒ **TRANSITION:** advance to the concrete example slide.

SLIDE 7 · Example of Directory Evaluation

timing: ∼70 sec

ON SCREEN: *Two files* f_1, f_2 *built via* protect(mkfile n c) p. *Two-entry directory* d. *Values of* `contains` *at* n_1, n_2, n_3 *and* `size` $d = 2$.

"To close out, a concrete evaluation."

→ **Point to f_1 and f_2 definitions on the slide.**

"Let f_1 *be* protect *applied to* mkfile n_1 c_1 *with* ***read*** *permission. Let* f_2 *be* protect *applied to* mkfile n_2 c_2 *with* ***write*** *permission. Let* d *be* addfile f_1 (addfile f_2 emptydir) *— a two-entry directory."*

→ **Point to each `contains` equation.**

"`contains` n_1 d *is* ***true*** *—* f_1*'s name matches at the outer layer.* `contains` n_2 d *is also* ***true*** *— we recurse into the tail and find* f_2*.* `contains` n_3 d *is* ***false*** *— no match at either layer."*

→ **Point to the `size` line.**

"And `size` d *equals* ***two*** *— each* addfile *contributes one,* emptydir *contributes zero. So the two-entry directory has size two, confirming Theorem 5."*

"That completes my sections of the development. Hamza will now take over with the theory morphism."

⇒ **HAND OFF:** look to Hamza, step slightly aside, smile.

Q&A PREP · Likely Questions

rehearse these — one of them almost certainly comes up

Q1. Why flat directories and not hierarchical?

"Scope — the project name says ***simple*** *file system. Flat keeps the inductive type within the standard fourth-kind constructor pattern of Chapter 10, preserves the direct list analogy, and makes the FILE-to-DIR morphism clean because single-entry directories give a canonical target for the embedding. Hierarchy would require a second recursive constructor* $\mathrm{adddir} : D \to D \to D$*, which breaks the clean list analogy and makes the morphism extraction ambiguous. We note hierarchy as future work in the paper."*

Q2. Why are `contains` and `size` in a development instead of extending DIRECTORY?

"Because they're ***derived*** *operations — they're definable from the primitives* addfile *and* emptydir*, so postulating them as theory constants would add redundant axioms. The development framework lets us define them once, prove they exist uniquely, and then use them. In Project 1 we put derived operations into theory extensions and lost points for it — this is the direct correction."*

Q3. How did you verify the DIRECTORY obligations?

"All seven obligations from Chapter 10 of Farmer's textbook — non-empty quasitypes, totality on domain, injectivity, disjointness of ranges, structural induction. Each one discharges directly from one of our four axioms, Axioms 13 through 16, with the non-emptiness obligation for the F*-argument holding because* F *is a base type and every base type is inhabited in any Alonzo frame. The full check is in the paper."*

Q4. What if two files in a directory have the same name?

"They're allowed. The inductive type doesn't enforce uniqueness — you can have two addfile *applications with files sharing a name.* `contains` *would just return true if* ***any*** *match is found. A future extension could*

add a uniqueness invariant, but that's a development-layer property, not a theory-layer one."

Q5. Why use K_D for the constructor set?

"Standard inductive-type convention — K for Konstruktor, subscripted by the type. The subscript makes it unambiguous which type's constructors we mean, and it avoids clashing with the base type C of FILE."

Q6. What does "pushout-style construction" mean?

"It's a way of combining two theories that share no non-trivial subtheory. SECURED-FILE takes FILE and PERM as inputs and produces a new theory whose language is the union of both, augmented with the new constants perm *and* protect*. The category-theoretic term is a pushout over the empty theory. In Alonzo's little theories method, we encode it by having SECURED-FILE extend both parents simultaneously, which is what the two arrows in our theory graph represent."*